

Unit Test Documentation for test_game_logic.cpp

1. Introduction

This document describes the purpose, structure, and usage of the unit testing code implemented using the Google Test framework for the Tic Tac Toe game project. The testing is focused on validating the core logic and utility functions of the game. All tests are located in the file test_game_logic.cpp.

2. Purpose of the Tests

The tests are designed to automatically verify the correctness of:

- Game outcome detection (win, draw)
- AI move generation
- Utility functions (hashing and input escaping)

These tests help ensure that the game behaves as expected and make the codebase easier to maintain and extend.

3. Google Test Overview

Google Test is a C++ testing framework developed by Google. It provides tools to write and run repeatable, structured test cases. It supports:

- Test assertions (EXPECT_EQ, ASSERT_TRUE, etc.)
- Test fixtures for shared setup and teardown
- Detailed output showing passed and failed test cases.

4. Test File Overview (test_game_logic.cpp)

Header and Setup:

- #define TESTING ensures the main() function from tictactoe.cpp is excluded from compilation.
- Game logic is imported for testing.
- gtest/gtest.h is included to access Google Test functionality.

Test Fixture Class:

- TicTacToeTest uses Google Test's Test fixture class.
- It resets the game board before each test case to avoid interference between tests.

5. Test Categories and Examples

A. Game Winner Detection (checkWinner)

- NoWinnerInitially: Empty board = no winner
- RowWinX: First row full of 'X' = X wins
- ColumnWinO: One column full of 'O' = O wins
- DiagonalWinX: Top-left to bottom-right diagonal = X wins
- AntiDiagonalWinO: Top-right to bottom-left diagonal = O wins

B. Draw Detection (isDraw)

- NotDrawIfEmpty: Empty board is not a draw
- DrawScenario: Full board without winner is a draw

C. AI Move Validity (easyAIMove)

Unit Test Documentation for test_game_logic.cpp

- EasyAIMovesValid: AI picks a valid empty cell

D. Utility Functions

- simpleHashConsistency: Same input = same hash; different input = different hash
- EscapeSingleQuote: Ensures single quotes are properly escaped

6. Main Function for Test Execution

```
int main(int argc, char **argv) {  
    ::testing::InitGoogleTest(&argc, argv);  
    return RUN_ALL_TESTS();  
}
```

This function initializes the Google Test framework and runs all defined test cases.

7. How to Compile and Run the Tests

CMake Setup:

```
add_executable(test_game tests/test_game_logic.cpp src/tictactoe.cpp)  
target_link_libraries(test_game gtest)
```

Build with:

```
cmake -G "MinGW Makefiles" ..  
mingw32-make
```

Run with:

```
./test_game.exe
```

8. Conclusion

This unit test file plays a crucial role in verifying the game's internal logic. Using Google Test ensures that all core functions of the Tic Tac Toe game are tested under multiple conditions. It improves code reliability, maintainability, and enables quick validation of changes during future development.